

Commissioning Platform — Implementation Guide

Pinned Dependencies

```
# requirements.txt
pymodbus==3.7.4
BAC0==24.3.25
pysnmp-lexstudio==6.1.2
pynvml==12.560.30
influxdb-client[async]==1.44.0
minio==7.2.12
fastapi==0.115.6
uvicorn[standard]==0.34.0
sse-starlette==2.1.3
httpx==0.27.2
ifcopenshell==0.8.1
pyyaml==6.0.2
aiofiles==24.1.0
python-multipart==0.0.18
reportlab==4.2.5
anthropic==0.39.0
```

// frontend/package.json dependencies

```
{
  "react": "^18.3.0",
  "react-dom": "^18.3.0",
  "react-router-dom": "^6.28.0"
}
```

Project Structure

```
/app/
├── config/
│   └── platform.yml
├── data/
│   └── wal/                                # Write-ahead log for WORM failures
├── src/
│   ├── __init__.py
│   ├── main.py                            # Entry point. Starts asyncio loop.
│   ├── config.py                          # Loads platform.yml
│   ├── types.py                           # All dataclasses (shared types)
│   ├── config_loader.py                   # Module 1
│   ├── netbox_reader.py                   # Module 2
│   └── pollers/
│       ├── __init__.py                    # Module 3
│       ├── modbus.py                      # Module 7
│       ├── bacnet.py                      # Module 7
│       └── snmp.py                        # Module 8
```

- └─ nvml_poller.py # Module 9
- └─ influx_writer.py # Module 4
- └─ attestation.py # Module 5
- └─ workers/
 - └─ __init__.py
 - └─ modbus_worker.py # Module 6
 - └─ bacnet_worker.py # (same pattern as modbus)
 - └─ snmp_worker.py
 - └─ nvml_worker.py
- └─ test_engine.py # Module 10
- └─ discovery.py # Module 11
- └─ connectors/
 - └─ __init__.py
 - └─ genetec.py # Module 12
 - └─ servicenow.py
 - └─ maximo.py
- └─ orchestrator.py # Module 13
- └─ reconciliation.py # Module 14
- └─ reports.py # Module 15
- └─ bim_import.py # Module 16
- └─ pdf_pipeline.py # Module 17
- └─ api/
 - └─ __init__.py
 - └─ server.py # Module 21
- └─ frontend/ # Modules 18-20
 - └─ public/
 - └─ index.html
 - └─ src/
 - └─ App.jsx
 - └─ index.jsx
 - └─ context/
 - └─ FacilityContext.jsx
 - └─ hooks/
 - └─ useApi.js
 - └─ useSSE.js
 - └─ pages/
 - └─ Dashboard.jsx # Module 18
 - └─ DeviceList.jsx
 - └─ DeviceDetail.jsx
 - └─ TestExecution.jsx
 - └─ TestDetail.jsx
 - └─ PunchList.jsx
 - └─ Checklist.jsx # Module 19
 - └─ Attestation.jsx # Module 20
 - └─ Reports.jsx
 - └─ Import.jsx
 - └─ components/
 - └─ PowerDepGraph.jsx
 - └─ LiveRegisters.jsx
- └─ simulator/

└─ sim.py	# Module 22
└─ docker-compose.yml	# Module 0
└─ Dockerfile	
└─ requirements.txt	

Module 0: Docker Compose

File: /app/docker-compose.yml

No code to write. YAML only. Key points:

- Platform container uses `network_mode: host` to reach facility devices.
- All other containers on `cx-net` bridge network.
- Platform container connects to both: `--network cx-net` won't work with `network_mode: host`.
- Solution: platform reaches infrastructure containers via `localhost` published ports (8000, 8086, 9000). Reaches facility devices via host network directly.

MinIO bucket creation with Object Lock (run once after first `docker compose up`):

```
mc alias set local http://localhost:9000 ACCESS_KEY SECRET_KEY
mc mb local/attestation --with-lock
mc retention set --default COMPLIANCE 2555d local/attestation
```

Shared Types: src/types.py

Every module imports from this file. Write this FIRST before any module.

```
from dataclasses import dataclass, field
from typing import Optional
import time
import uuid

@dataclass
class DeviceInfo:
    device_id: str
    name: str
    primary_ip: str
    device_type_slug: str
    config_context: dict
    protocol: str  # extracted from
    config_context["protocol"]
    site: str
    rack: str
    position: int

@dataclass
```

```

class PollResult:
    device_id: str
    timestamp_ns: int          # time.time_ns()
    measurements: dict         # {"register_name": float_value}
    raw_bytes: dict           # {"register_name": "hex_string"}
    protocol: str
    source_ip: str
    success: bool
    errors: dict = field(default_factory=dict) # {"register_name":
"error msg"}

```

@dataclass

```

class AttestationRecord:
    timestamp_ns: int
    device_id: str
    measurement: str
    value: float
    raw_bytes: str
    protocol: str
    source_ip: str
    worker_id: str
    test_id: Optional[str] = None
    facility: str = ""
    nonce: str = ""

```

@dataclass

```

class AttestedRecord(AttestationRecord):
    hash: str = ""
    previous_hash: str = ""
    chain_id: str = ""
    sequence: int = 0

```

@dataclass

```

class TestRequest:
    test_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    device_id: str = ""
    test_name: str = ""
    requested_by: str = ""
    requested_at: str = ""
    priority: int = 3

```

@dataclass

```

class TestResult:
    test_id: str = ""
    device_id: str = ""
    test_name: str = ""
    status: str = "" # passed|failed|aborted|restore_failure|
precondition_failed|manual_pending
    started_at: str = ""
    completed_at: str = ""

```

```

duration_seconds: float = 0.0
precondition_results: list = field(default_factory=list)
manual_confirmations: list = field(default_factory=list)
acceptance_results: list = field(default_factory=list)
abort_triggered: bool = False
abort_reason: Optional[str] = None
restore_success: bool = True
evidence_hashes: list = field(default_factory=list)
contact_wear_before: Optional[float] = None
contact_wear_after: Optional[float] = None

@dataclass
class PunchListItem:
    id: str = field(default_factory=lambda: str(uuid.uuid4()))
    severity: str = "" # critical|major|minor|info
    category: str = "" # identity|firmware|network|power|
    cooling|security|safety|test_failure|physical
    device_id: str = ""
    device_name: str = ""
    site: str = ""
    rack: str = ""
    expected: str = ""
    actual: str = ""
    source: str = "" # discovery|passive_read|active_test|
    reconciliation|physical_checklist
    evidence_hash: Optional[str] = None
    test_id: Optional[str] = None
    remediation: str = ""
    status: str = "open" # open|acknowledged|resolved|deferred
    resolved_by: Optional[str] = None
    resolved_at: Optional[str] = None

@dataclass
class Event:
    event_type: str # device_discovered|poll_result|test_started|
    test_completed|...
    timestamp: str # ISO 8601
    data: dict = field(default_factory=dict)

```

Shared Config: src/config.py

```

import yaml
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class PlatformConfig:
    facility_name: str
    scan_subnets: List[str]

```

```

netbox_url: str
netbox_token: str
influxdb_url: str
influxdb_token: str
influxdb_org: str
influxdb_bucket: str
minio_endpoint: str
minio_access_key: str
minio_secret_key: str
minio_bucket: str
minio_retention_days: int
max_concurrent_polls: int
max_concurrent_tests: int
api_port: int
cors_origins: List[str]
ntp_server: str
log_level: str
external_apis: dict

```

```

def load_config(path: str = "/app/config/platform.yml") ->
PlatformConfig:
    with open(path) as f:
        raw = yaml.safe_load(f)
    return PlatformConfig(
        facility_name=raw["facility"]["name"],
        scan_subnets=raw["facility"]["scan_subnets"],
        netbox_url=raw["netbox"]["url"],
        netbox_token=raw["netbox"]["token"],
        influxdb_url=raw["influxdb"]["url"],
        influxdb_token=raw["influxdb"]["token"],
        influxdb_org=raw["influxdb"]["org"],
        influxdb_bucket=raw["influxdb"].get("bucket",
"commissioning"),
        minio_endpoint=raw["minio"]["endpoint"],
        minio_access_key=raw["minio"]["access_key"],
        minio_secret_key=raw["minio"]["secret_key"],
        minio_bucket=raw["minio"].get("bucket", "attestation"),
        minio_retention_days=raw["minio"].get("retention_days", 2555),

max_concurrent_polls=raw["workers"].get("max_concurrent_polls", 50),

max_concurrent_tests=raw["orchestrator"].get("max_concurrent_tests",
5),

        api_port=raw["api"].get("port", 8080),
        cors_origins=raw["api"].get("cors_origins",
["http://localhost:8080"]),
        ntp_server=raw.get("ntp", {}).get("server", "pool.ntp.org"),
        log_level=raw.get("logging", {}).get("level", "INFO"),
        external_apis=raw.get("external_apis", {}),
    )

```

Module 1: Config Context Loader — src/config_loader.py

```
import json
import httpx

async def load_config_context(
    netbox_url: str,
    netbox_token: str,
    json_path: str,
    device_type_slug: str
) -> dict:
    """Load a Config Context JSON file into NetBox for a device
    type."""

    # 1. Read JSON file
    with open(json_path) as f:
        config_data = json.load(f)

    # 2. Get device type ID from NetBox
    headers = {"Authorization": f"Token {netbox_token}"}
    async with httpx.AsyncClient(base_url=netbox_url) as client:
        resp = await client.get(
            "/api/dcim/device-types/",
            params={"slug": device_type_slug},
            headers=headers
        )
        resp.raise_for_status()
        results = resp.json()["results"]
        if not results:
            raise ValueError(f"Device type '{device_type_slug}' not
found in NetBox")
        device_type_id = results[0]["id"]

    # 3. Create Config Context
    payload = {
        "name": f"{device_type_slug}-config",
        "weight": 1000,
        "data": config_data,
        "device_types": [device_type_id],
        "is_active": True,
    }
    resp = await client.post(
        "/api/extras/config-contexts/",
        json=payload,
        headers=headers
    )
    resp.raise_for_status()
    return resp.json()
```

Test: await load_config_context("http://localhost:8000", "token", "cm2000.json", "cm2000") Then: GET /api/dcim/devices/{id}/ and verify config_context field contains the registers.

Module 2: NetBox Device Reader — src/netbox_reader.py

```
import httpx
from .types import DeviceInfo

async def get_devices_by_protocol(
    netbox_url: str,
    netbox_token: str,
    protocol: str
) -> list[DeviceInfo]:
    """Query NetBox for all active devices whose Config Context
    specifies the given protocol."""

    headers = {"Authorization": f"Token {netbox_token}"}
    devices = []

    async with httpx.AsyncClient(base_url=netbox_url) as client:
        url = "/api/dcim/devices/"
        params = {"status": "active", "limit": 1000}

        while url:
            resp = await client.get(url, params=params,
headers=headers)
            resp.raise_for_status()
            data = resp.json()

            for d in data["results"]:
                ctx = d.get("config_context", {})
                if ctx.get("protocol") == protocol:
                    devices.append(DeviceInfo(
                        device_id=str(d["id"]),
                        name=d["name"],
                        primary_ip=(d.get("primary_ip4") or
{}).get("address", "").split("/")[0]
                        or ctx.get("connection", {}).get("host",
""),

                        device_type_slug=d["device_type"]["slug"],
                        config_context=ctx,
                        protocol=protocol,
                        site=d.get("site", {}).get("name", ""),
                        rack=(d.get("rack") or {}).get("name", ""),
                        position=d.get("position") or 0,
                    ))

            url = data.get("next") # pagination
```



```
        params = {} # params already in next URL

    return devices
```

Module 3: Modbus TCP Poller — src/pollers/modbus.py

```
from pymodbus.client import AsyncModbusTcpClient
from pymodbus.payload import BinaryPayloadDecoder
from pymodbus.constants import Endian
import time
from ..types import DeviceInfo, PollResult

# Endian mapping from Config Context strings to pymodbus constants
ENDIAN_MAP = {"big": Endian.BIG, "little": Endian.LITTLE}

# Decoder method mapping from Config Context data_type strings
DECODE_MAP = {
    "uint16": lambda d: d.decode_16bit_uint(),
    "int16": lambda d: d.decode_16bit_int(),
    "uint32": lambda d: d.decode_32bit_uint(),
    "int32": lambda d: d.decode_32bit_int(),
    "float32": lambda d: d.decode_32bit_float(),
    "float64": lambda d: d.decode_64bit_float(),
    "boolean": lambda d: bool(d.decode_16bit_uint()),
}

async def poll_device(device: DeviceInfo) -> PollResult:
    """Poll all registers defined in the device's Config Context."""

    conn = device.config_context.get("connection", {})
    host = device.primary_ip
    port = conn.get("port", 502)
    unit_id = conn.get("unit_id", 1)
    byte_order = ENDIAN_MAP.get(conn.get("byte_order", "big"),
    Endian.BIG)
    word_order = ENDIAN_MAP.get(conn.get("word_order", "big"),
    Endian.BIG)
    timeout = conn.get("timeout_seconds", 5)
    retries = conn.get("retries", 3)

    measurements = {}
    raw_bytes = {}
    errors = {}

    client = AsyncModbusTcpClient(
        host=host,
        port=port,
        timeout=timeout,
        retries=retries,
```

```

        retry_on_empty=True,
    )

    try:
        connected = await client.connect()
        if not connected:
            return PollResult(
                device_id=device.device_id,
timestamp_ns=time.time_ns(),
                measurements={}, raw_bytes={}, protocol="modbus_tcp",
                source_ip=host, success=False,
                errors={"_connection": f"Failed to connect to {host}:"
{port}}})
    )

    for reg in device.config_context.get("registers", []):
        name = reg["name"]
        address = reg["address"]
        count = reg.get("count", 1)
        fc = reg.get("function_code", 3)
        data_type = reg["data_type"]
        scale = reg.get("scale", 1.0)

        try:
            if fc == 3:
                result = await client.read_holding_registers(
                    address=address, count=count, slave=unit_id
                )
            elif fc == 4:
                result = await client.read_input_registers(
                    address=address, count=count, slave=unit_id
                )
            else:
                errors[name] = f"Unsupported function code: {fc}"
                continue

            if result.isError():
                errors[name] = f"Modbus error: {result}"
                continue

            # Store raw bytes as hex
            raw_hex = "".join(f"{r:04x}" for r in
result.registers)
            raw_bytes[name] = raw_hex

            # Decode
            decoder = BinaryPayloadDecoder.fromRegisters(
                result.registers,
                byteorder=byte_order,
                wordorder=word_order

```

```

    )
    decode_fn = DECODER_MAP.get(data_type)
    if not decode_fn:
        errors[name] = f"Unknown data_type: {data_type}"
        continue

    value = decode_fn(decoder)

    # Check for NaN/Inf (byte order mismatch indicator)
    import math
    if isinstance(value, float) and (math.isnan(value) or
math.isinf(value)):
        errors[name] = f"NaN/Inf detected. Likely
byte/word order mismatch. Got raw: {raw_hex}"
        continue

    measurements[name] = value * scale

except Exception as e:
    errors[name] = str(e)

finally:
    client.close()

return PollResult(
    device_id=device.device_id,
    timestamp_ns=time.time_ns(),
    measurements=measurements,
    raw_bytes=raw_bytes,
    protocol="modbus_tcp",
    source_ip=host,
    success=len(errors) == 0,
    errors=errors,
)

```

Key: this is the exact PyModbus 3.7 API. AsyncModbusTcpClient, read_holding_registers(address=, count=, slave=), BinaryPayloadDecoder.fromRegisters(registers, byteorder=, wordorder=). Not the old ModbusTcpClient sync API. Not the old unit= parameter (renamed to slave= in 3.x).

Module 4: InfluxDB Writer — src/influx_writer.py

```

from influxdb_client.client.influxdb_client_async import
InfluxDBClientAsync
from influxdb_client import Point, WritePrecision
from ..types import PollResult

class InfluxWriter:

```

```

    def __init__(self, url: str, token: str, org: str, bucket: str):
        self.client = InfluxDBClientAsync(url=url, token=token,
org=org)
        self.bucket = bucket
        self.org = org

    async def write_poll(self, result: PollResult, test_id: str =
None) -> None:
        measurement = "test" if test_id else "poll"
        write_api = self.client.write_api()

        points = []
        for name, value in result.measurements.items():
            pt = (Point(measurement)
                .tag("device_id", result.device_id)
                .tag("protocol", result.protocol)
                .tag("source_ip", result.source_ip)
                .field(name, float(value))
                .time(result.timestamp_ns, WritePrecision.NS))
            if test_id:
                pt = pt.tag("test_id", test_id)
            points.append(pt)

        await write_api.write(bucket=self.bucket, org=self.org,
record=points)

    async def query_register_history(
        self, device_id: str, register: str,
        start: str, stop: str
    ) -> list[dict]:
        query_api = self.client.query_api()
        query = f'''
from(bucket: "{self.bucket}")
  |> range(start: {start}, stop: {stop})
  |> filter(fn: (r) => r["device_id"] == "{device_id}")
  |> filter(fn: (r) => r["_field"] == "{register}")
  |> sort(columns: ["_time"])
'''
        tables = await query_api.query(query, org=self.org)
        return [
            {"timestamp": r.get_time().isoformat(), "value":
r.get_value()}
            for table in tables for r in table.records
        ]

    async def close(self):
        await self.client.close()

```

Key: uses InfluxDBClientAsync not InfluxDBClient. The async version is in `influxdb_client.client.influxdb_client_async`. Write API from async client. Flux query language for reads.

Module 5: Attestation Engine — `src/attestation.py`

```
import asyncio
import hashlib
import json
import time
import os
import aiofiles
from minio import Minio
from datetime import timedelta
import io
from ..types import AttestationRecord, AttestedRecord

class AttestationEngine:
    def __init__(self, config):
        self.config = config
        self.minio = Minio(
            config.minio_endpoint,
            access_key=config.minio_access_key,
            secret_key=config.minio_secret_key,
            secure=False, # on-prem, no TLS to MinIO
        )
        self.queue = asyncio.Queue()
        self.previous_hash = "0" * 64 # genesis hash
        self.sequence = 0
        self.chain_id = config.facility_name
        self.worker_id = f"{os.uname().nodename}-{os.getpid()}"
        self._nonce_counter = 0
        self.wal_dir = "/app/data/wal"
        os.makedirs(self.wal_dir, exist_ok=True)

    def _canonical_json(self, record: AttestationRecord, prev_hash:
str) -> str:
        """Deterministic JSON for hashing. Sorted keys, no
whitespace."""
        obj = {
            "device_id": record.device_id,
            "measurement": record.measurement,
            "previous_hash": prev_hash,
            "raw_bytes": record.raw_bytes,
            "timestamp_ns": record.timestamp_ns,
            "value": record.value,
        }
        return json.dumps(obj, sort_keys=True, separators=(",", ":"))
```

```

def _compute_hash(self, canonical: str) -> str:
    return hashlib.sha256(canonical.encode("utf-8")).hexdigest()

async def submit(self, record: AttestationRecord) -> None:
    """Submit a record for attestation. Non-blocking."""
    record.facility = self.config.facility_name
    record.worker_id = self.worker_id
    self._nonce_counter += 1
    record.nonce = f"{self.worker_id}-{self._nonce_counter}"
    await self.queue.put(record)

async def run(self) -> None:
    """Main loop. Consumes from queue, hashes, writes to WORM."""
    while True:
        record = await self.queue.get()
        await self._process(record)

async def _process(self, record: AttestationRecord) -> None:
    canonical = self._canonical_json(record, self.previous_hash)
    hash_value = self._compute_hash(canonical)

    attested = AttestedRecord(
        **{k: v for k, v in record.__dict__.items()},
        hash=hash_value,
        previous_hash=self.previous_hash,
        chain_id=self.chain_id,
        sequence=self.sequence + 1,
    )

    serialized = json.dumps(attested.__dict__, default=str)
    date_str = time.strftime("%Y-%m-%d",
        time.gmtime(record.timestamp_ns / 1e9))
    key =
    f"{self.chain_id}/{date_str}/{attested.sequence:010d}.json"

    try:
        data = serialized.encode("utf-8")
        self.minio.put_object(
            bucket_name=self.config.minio_bucket,
            object_name=key,
            data=io.BytesIO(data),
            length=len(data),
            content_type="application/json",
            retention=None, # bucket-level default retention
        )
        # Success: advance chain
        self.previous_hash = hash_value
        self.sequence += 1
    except Exception as e:

```

applies

```

        # WORM failure: write to WAL
        wal_path = os.path.join(self.wal_dir,
f"{record.timestamp_ns}.json")
        async with aiofiles.open(wal_path, "w") as f:
            await f.write(serialized)
        # Do NOT advance previous_hash – chain pauses

    async def wal_recovery_loop(self) -> None:
        """Every 60s, retry WAL files."""
        while True:
            await asyncio.sleep(60)
            for fname in sorted(os.listdir(self.wal_dir)):
                fpath = os.path.join(self.wal_dir, fname)
                try:
                    async with aiofiles.open(fpath, "r") as f:
                        data = await f.read()
                        record_dict = json.loads(data)
                        key =
f"{record_dict['chain_id']}/{fname.split('.')[0]}/{record_dict['sequen
ce']:010d}.json"
                        encoded = data.encode("utf-8")
                        self.minio.put_object(
                            bucket_name=self.config.minio_bucket,
                            object_name=key,
                            data=io.BytesIO(encoded),
                            length=len(encoded),
                            content_type="application/json",
                        )
                        os.remove(fpath)
                except Exception:
                    pass # retry next cycle

    async def verify_chain(self, start_seq: int, count: int) -> dict:
        """Verify chain integrity for a range of records."""
        prefix = f"{self.chain_id}/"
        objects =
list(self.minio.list_objects(self.config.minio_bucket, prefix=prefix))
        # Sort by sequence, verify each hash
        breaks = []
        prev_hash = None
        for obj in sorted(objects, key=lambda o: o.object_name):
            data = self.minio.get_object(self.config.minio_bucket,
obj.object_name)
            record = json.loads(data.read())
            if prev_hash is not None and record["previous_hash"] !=
prev_hash:
                breaks.append(record["sequence"])
        # Recompute hash
        canonical = json.dumps({
            "device_id": record["device_id"],

```

```

        "measurement": record["measurement"],
        "previous_hash": record["previous_hash"],
        "raw_bytes": record["raw_bytes"],
        "timestamp_ns": record["timestamp_ns"],
        "value": record["value"],
    }, sort_keys=True, separators=(",", ":"))
    recomputed =
hashlib.sha256(canonical.encode()).hexdigest()
    if recomputed != record["hash"]:
        breaks.append(record["sequence"])
    prev_hash = record["hash"]
    return {"chain_valid": len(breaks) == 0, "breaks": breaks}

```

Key: minio-py put_object takes data as a BytesIO, not raw bytes. Must wrap with `io.BytesIO(data)` and pass length. Retention is set at bucket level via `mc retention set`, not per-object in code for simplicity.

Module 6: Modbus Worker — `src/workers/modbus_worker.py`

```

import asyncio
from ..netbox_reader import get_devices_by_protocol
from ..pollers.modbus import poll_device
from ..influx_writer import InfluxWriter
from ..attestation import AttestationEngine
from ..types import AttestationRecord, Event

class ModbusWorker:
    def __init__(self, config, influx: InfluxWriter, attestation:
AttestationEngine, event_queue: asyncio.Queue):
        self.config = config
        self.influx = influx
        self.attestation = attestation
        self.event_queue = event_queue
        self.devices = []
        self.failures = {} # device_id -> consecutive_failure_count

    async def refresh_devices(self):
        self.devices = await get_devices_by_protocol(
            self.config.netbox_url, self.config.netbox_token,
"modbus_tcp"
        )

    async def run(self):
        await self.refresh_devices()
        while True:
            tasks = []
            sem = asyncio.Semaphore(self.config.max_concurrent_polls)
            for device in self.devices:
                tasks.append(self._poll_with_sem(sem, device))

```



```

        await asyncio.gather(*tasks)

        # Sleep until next poll cycle (use shortest poll interval
across devices)
        min_interval = min(
            (d.config_context.get("poll_interval_seconds", 30) for
d in self.devices),
            default=30
        )
        await asyncio.sleep(min_interval)

    async def _poll_with_sem(self, sem, device):
        async with sem:
            result = await poll_device(device)

            if not result.success:
                self.failures[device.device_id] =
self.failures.get(device.device_id, 0) + 1
                if self.failures[device.device_id] >= 3:
                    await self.event_queue.put(Event(
                        event_type="worker_health",
                        timestamp="",
                        data={"device_id": device.device_id, "status":
"UNREACHABLE"}
                    ))
                return

            self.failures[device.device_id] = 0

        # Write to InfluxDB
        await self.influx.write_poll(result)

        # Submit each measurement for attestation
        for name, value in result.measurements.items():
            await self.attestation.submit(AttestationRecord(
                timestamp_ns=result.timestamp_ns,
                device_id=result.device_id,
                measurement=name,
                value=value,
                raw_bytes=result.raw_bytes.get(name, ""),
                protocol=result.protocol,
                source_ip=result.source_ip,
                worker_id="", # filled by attestation engine
            ))

        # Push event for dashboard
        await self.event_queue.put(Event(
            event_type="poll_result",
            timestamp="",
            data={"device_id": device.device_id, "measurements":

```

```
result.measurements}
    ))
```

Modules 7, 8, 9 follow the exact same pattern. Replace `poll_device` with the BAC0/PySNMP/pynvml equivalent. Replace `"modbus_tcp"` with `"bacnet_ip"`, `"snmp"`, `"nvml"`.

Module 7: BACnet/IP Poller — `src/pollers/bacnet.py`

```
import BAC0
import time
from ..types import DeviceInfo, PollResult

# BAC0 must be initialized ONCE per process
_bacnet = None

def get_bacnet():
    global _bacnet
    if _bacnet is None:
        _bacnet = BAC0.lite() # lightweight BACnet stack, no device
    creation
    return _bacnet

async def poll_device(device: DeviceInfo) -> PollResult:
    bacnet = get_bacnet()
    measurements = {}
    raw_bytes = {}
    errors = {}

    host = device.primary_ip
    for obj in device.config_context.get("objects", []):
        name = obj["name"]
        obj_type = obj["type"] # e.g. "analogInput"
        obj_instance = obj["instance"] # e.g. 1
        prop = obj.get("property", "presentValue")

        try:
            # BAC0 read format: "address objectType instance property"
            val = bacnet.read(f"{host} {obj_type} {obj_instance}
{prop}")
            measurements[name] = float(val)
            raw_bytes[name] = str(val)
        except Exception as e:
            errors[name] = str(e)

    return PollResult(
        device_id=device.device_id,
        timestamp_ns=time.time_ns(),
        measurements=measurements,
```

```

        raw_bytes=raw_bytes,
        protocol="bacnet_ip",
        source_ip=host,
        success=len(errors) == 0,
        errors=errors,
    )

```

Key: BAC0.lite() creates a lightweight BACnet stack. Must be initialized once. Read format is "IP objectType instance property". BAC0 handles the Who-Is, I-Am, and ReadProperty internally.

Warning: BAC0.read() is synchronous. Wrap in `asyncio.to_thread()` in the worker:
`result = await asyncio.to_thread(poll_device_sync, device).`

Module 8: SNMP Poller — `src/pollers/snmp.py`

```

from pynmp.hlapi.v3arch.asyncio import *
import time
from ..types import DeviceInfo, PollResult

async def poll_device(device: DeviceInfo) -> PollResult:
    measurements = {}
    raw_bytes = {}
    errors = {}

    host = device.primary_ip
    conn = device.config_context.get("connection", {})
    community = conn.get("community", "public")
    port = conn.get("port", 161)
    timeout_s = conn.get("timeout_seconds", 5)

    for oid_def in device.config_context.get("oids", []):
        name = oid_def["name"]
        oid = oid_def["oid"]
        scale = oid_def.get("scale", 1.0)

        try:
            errorIndication, errorStatus, errorIndex, varBinds = await
get_cmd(
            SnmpEngine(),
            CommunityData(community),
            await UdpTransportTarget.create((host, port),
timeout=timeout_s, retries=3),
            ContextData(),
            ObjectType(ObjectIdentity(oid))
        )

            if errorIndication:
                errors[name] = str(errorIndication)

```

```

        continue
    if errorStatus:
        errors[name] = f"SNMP error:
{errorStatus.prettyPrint()}"
        continue

    for varBind in varBinds:
        val = varBind[1]
        raw_bytes[name] = str(val)
        measurements[name] = float(val) * scale

    except Exception as e:
        errors[name] = str(e)

    return PollResult(
        device_id=device.device_id,
        timestamp_ns=time.time_ns(),
        measurements=measurements,
        raw_bytes=raw_bytes,
        protocol="snmp",
        source_ip=host,
        success=len(errors) == 0,
        errors=errors,
    )

```

Key: pysnmp-lexstudio 6.x API. Uses `pysnmp.hlapi.v3arch.asyncio` not the old `pysnmp.hlapi.asyncio`. `UdpTransportTarget.create()` is async and must be awaited. This is the most common mistake with the new API.

Module 10: Active Test Engine — `src/test_engine.py`

```

import asyncio
import time
from datetime import datetime
from .pollers.modbus import poll_device as modbus_poll
from .types import TestRequest, TestResult, AttestationRecord,
PunchListItem, Event, DeviceInfo

class TestEngine:
    def __init__(self, config, influx, attestation, event_queue):
        self.config = config
        self.influx = influx
        self.attestation = attestation
        self.event_queue = event_queue
        self.manual_confirmations = {} # test_id -> asyncio.Event

    async def execute(self, request: TestRequest, device: DeviceInfo)
-> TestResult:
        test_def = None

```

```

    for t in device.config_context.get("active_tests", []):
        if t["name"] == request.test_name:
            test_def = t
            break
    if not test_def:
        return TResult(test_id=request.test_id,
status="failed",
                                abort_reason=f"Test
'{request.test_name}' not found in Config Context")

    result = TResult(
        test_id=request.test_id,
        device_id=request.device_id,
        test_name=request.test_name,
        started_at=datetime.utcnow().isoformat() + "Z",
    )

    # --- PRECONDITION CHECK ---
    poll = await modbus_poll(device)
    for pre in test_def.get("preconditions", []):
        reg_name = pre["register"]
        operator = pre["operator"]
        expected = pre["value"]
        actual = poll.measurements.get(reg_name)
        passed = self._evaluate(actual, operator, expected)
        result.precondition_results.append({
            "register": reg_name, "expected": expected,
            "actual": actual, "passed": passed
        })
    if not passed:
        result.status = "precondition_failed"
        result.completed_at = datetime.utcnow().isoformat() +
"Z"

        return result

    # --- MANUAL CONFIRMATION ---
    for prompt in test_def.get("manual_confirmation", []):
        result.status = "manual_pending"
        await self.event_queue.put(Event(
            event_type="manual_confirmation_needed",
            timestamp=datetime.utcnow().isoformat() + "Z",
            data={"test_id": request.test_id, "prompt": prompt}
        ))
        confirm_event = asyncio.Event()
        self.manual_confirmations[request.test_id] = confirm_event
        try:
            await asyncio.wait_for(confirm_event.wait(),
timeout=1800) # 30 min
        except asyncio.TimeoutError:
            result.status = "aborted"

```

```

        result.abort_reason = "Manual confirmation timed out"
        result.completed_at = datetime.utcnow().isoformat() +
"Z"

        return result

# --- IDENTITY VERIFICATION ---
identity_reg = device.config_context.get("identity_register")
if identity_reg:
    id_poll = await modbus_poll(device)
    actual_id =
id_poll.measurements.get(identity_reg.get("register"))
    expected_id = identity_reg.get("expected")
    if actual_id != expected_id:
        result.status = "aborted"
        result.abort_reason = f"Identity mismatch: expected
{expected_id}, got {actual_id}"
        result.completed_at = datetime.utcnow().isoformat() +
"Z"

        return result

# --- COMMAND ---
from pymodbus.client import AsyncModbusTcpClient
conn = device.config_context.get("connection", {})
client = AsyncModbusTcpClient(host=device.primary_ip,
port=conn.get("port", 502))
await client.connect()

cmd = test_def["command"]
if cmd["function_code"] == 6:
    await client.write_register(
        address=cmd["address"], value=int(cmd["value"]),
        slave=conn.get("unit_id", 1)
    )
elif cmd["function_code"] == 16:
    await client.write_registers(
        address=cmd["address"], values=[int(cmd["value"])],
        slave=conn.get("unit_id", 1)
    )

await self.event_queue.put(Event(
    event_type="test_started", timestamp="",
    data={"test_id": request.test_id, "device_id":
device.device_id}
))

# --- MONITORING ---
interval_ms = test_def.get("monitor_interval_ms", 1000)
duration_s = test_def.get("monitor_duration_seconds", 60)
monitor_names = test_def.get("monitor", [])
abort_conditions = test_def.get("abort_conditions", [])

```

```

collected = [] # list of PollResult
start = time.time()
consecutive_failures = 0

while (time.time() - start) < duration_s:
    poll = await modbus_poll(device)

    if not poll.success:
        consecutive_failures += 1
        if consecutive_failures >= 3:
            result.abort_triggered = True
            result.abort_reason = "device_unreachable"
            break
        await asyncio.sleep(interval_ms / 1000)
        continue

    consecutive_failures = 0
    collected.append(poll)

# Attest each measurement
    for name in monitor_names:
        if name in poll.measurements:
            await self.attestation.submit(AttestationRecord(
                timestamp_ns=poll.timestamp_ns,
                device_id=device.device_id,
                measurement=name,
                value=poll.measurements[name],
                raw_bytes=poll.raw_bytes.get(name, ""),
                protocol="modbus_tcp",
                source_ip=device.primary_ip,
                test_id=request.test_id,
            ))

# Write to InfluxDB
    await self.influx.write_poll(poll,
test_id=request.test_id)

# Check abort conditions
    for abort in abort_conditions:
        reg = abort["register"]
        val = poll.measurements.get(reg)
        if val is not None and self._evaluate(val,
abort["operator"], abort["value"]):
            result.abort_triggered = True
            result.abort_reason = f"{reg} {abort['operator']}"
            {abort['value']}
            break

    if result.abort_triggered:

```

```

        break

    await asyncio.sleep(interval_ms / 1000)

# --- RESTORE ---
restore = test_def.get("restore", {})
if restore:
    try:
        if restore.get("function_code", 6) == 6:
            await client.write_register(
                address=restore["address"],
value=int(restore["value"]),
                slave=conn.get("unit_id", 1)
            )
    except Exception:
        pass # blind restore on failure – safer to try than
not

# --- RESTORE VERIFY ---
restore_timeout = test_def.get("restore_timeout_seconds", 30)
restore_verify = test_def.get("restore_verify", [])
if restore_verify:
    rv_start = time.time()
    while (time.time() - rv_start) < restore_timeout:
        poll = await modbus_poll(device)
        all_ok = True
        for rv in restore_verify:
            val = poll.measurements.get(rv["register"])
            if not self._evaluate(val, rv["operator"],
rv["value"]):
                all_ok = False
                break
        if all_ok:
            result.restore_success = True
            break
        await asyncio.sleep(1)
    else:
        result.restore_success = False
        result.status = "restore_failure"

client.close()

# --- EVALUATE ACCEPTANCE ---
if not result.abort_triggered and result.status !=
"restore_failure":
    all_passed = True
    for acc in test_def.get("acceptance", []):
        reg = acc["register"]
        metric = acc["metric"]
        values = [p.measurements.get(reg) for p in collected

```



```

if reg in p.measurements]
    if not values:
        result.acceptance_results.append({
            "register": reg, "metric": metric,
            "expected": acc["value"], "actual": None,
            "passed": False
        })
        all_passed = False
        continue

    actual = self._compute_metric(metric, values,
        collected, reg, acc.get("value"))
    passed = self._evaluate(actual, acc["operator"],
        acc["value"])
    result.acceptance_results.append({
        "register": reg, "metric": metric,
        "expected": acc["value"], "actual": actual,
        "passed": passed
    })
    if not passed:
        all_passed = False

    result.status = "passed" if all_passed else "failed"

    if result.abort_triggered and result.status not in
("restore_failure",):
        result.status = "aborted"

    result.completed_at = datetime.utcnow().isoformat() + "Z"
    result.duration_seconds = (time.time() - start)

    await self.event_queue.put(Event(
        event_type="test_completed", timestamp="",
        data={"test_id": request.test_id, "status": result.status}
    ))

    return result

def _evaluate(self, actual, operator: str, expected) -> bool:
    if actual is None:
        return False
    ops = {
        "eq": lambda a, e: a == e,
        "neq": lambda a, e: a != e,
        "gt": lambda a, e: a > e,
        "lt": lambda a, e: a < e,
        "gte": lambda a, e: a >= e,
        "lte": lambda a, e: a <= e,
    }
    return ops.get(operator, lambda a, e: False)(actual, expected)

```

```

    def _compute_metric(self, metric, values, collected, reg,
threshold=None):
        if metric == "max":
            return max(values)
        elif metric == "min":
            return min(values)
        elif metric == "avg":
            return sum(values) / len(values)
        elif metric == "delta":
            return abs(values[-1] - values[0])
        elif metric == "settled_value":
            tail = values[-(len(values)//10 or 1):]
            return sum(tail) / len(tail)
        elif metric == "time_to_value":
            # Find first timestamp where value crossed threshold
            for i, p in enumerate(collected):
                v = p.measurements.get(reg)
                if v is not None and v >= threshold:
                    if i == 0:
                        return 0
                    dt = (p.timestamp_ns - collected[0].timestamp_ns)
/ 1e6 # ms
                    return dt
            return float("inf")
        elif metric == "rate_of_change":
            if len(values) < 2:
                return 0
            dt = (collected[-1].timestamp_ns -
collected[0].timestamp_ns) / 1e9
            if dt == 0:
                return 0
            return (values[-1] - values[0]) / dt
        return None

    def confirm_manual(self, test_id: str, confirmed_by: str):
        event = self.manual_confirmations.get(test_id)
        if event:
            event.set()

```

Module 21: FastAPI Server — src/api/server.py

Skeleton showing SSE and key routes:

```

from fastapi import FastAPI, Header, HTTPException, UploadFile, File
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware
from sse_starlette.sse import EventSourceResponse
import asyncio

```

```

import json

app = FastAPI()

# -- set in main.py after creating instances --
test_engine = None
orchestrator = None
attestation_engine = None
influx_writer = None
event_queue = None
config = None

def init(cfg, te, orch, att, influx, eq):
    global config, test_engine, orchestrator, attestation_engine,
    influx_writer, event_queue
    config, test_engine, orchestrator, attestation_engine,
    influx_writer, event_queue = \
        cfg, te, orch, att, influx, eq

# Auth
async def verify_key(x_api_key: str = Header(...)):
    if x_api_key != config.api_key:
        raise HTTPException(status_code=401)

# SSE endpoint
@app.get("/api/v1/system/events")
async def system_events():
    async def event_generator():
        local_queue = asyncio.Queue()
        # Fan-out: copy events to this client's queue
        # (In production, use a broadcast pattern)
        while True:
            try:
                event = await asyncio.wait_for(event_queue.get(),
                    timeout=30)
                yield {"event": event.event_type, "data":
                    json.dumps(event.data)}
            except asyncio.TimeoutError:
                yield {"event": "keepalive", "data": "{}"}
    return EventSourceResponse(event_generator())

# Test execution
@app.post("/api/v1/tests/run")
async def run_test(body: dict):
    from ..types import TestRequest
    req = TestRequest(device_id=body["device_id"],
        test_name=body["test_name"])
    await orchestrator.submit(req)
    return {"test_id": req.test_id, "status": "queued"}

```

```
@app.post("/api/v1/tests/confirm/{test_id}")
async def confirm_test(test_id: str, body: dict):
    test_engine.confirm_manual(test_id, body.get("confirmed_by",
"unknown"))
    return {"status": "confirmed"}
```

... remaining endpoints per Section 4 of contracts spec ...

Serve frontend

```
app.mount("/", StaticFiles(directory="/app/frontend/build",
html=True), name="frontend")
```

Key: sse-starlette provides EventSourceResponse. FastAPI has no built-in SSE. The `event_generator` is an async generator yielding dicts with `event` and `data` keys. The SSE spec requires `event:` and `data:` fields separated by newlines — `sse-starlette` handles this.

Key: event fan-out. The skeleton shows a simplified single-consumer pattern. For multiple dashboard clients, replace with a broadcast pattern: maintain a `set[asyncio.Queue]` of subscriber queues, each SSE connection registers its own queue, events are pushed to all queues.

Module 18: Dashboard — frontend/src/hooks/useSSE.js

```
import { useState, useEffect, useCallback } from "react";

export function useSSE(url) {
    const [events, setEvents] = useState([]);
    const [connected, setConnected] = useState(false);

    useEffect(() => {
        const source = new EventSource(url);

        source.onopen = () => setConnected(true);
        source.onerror = () => setConnected(false);

        source.addEventListener("poll_result", (e) => {
            setEvents((prev) => [...prev.slice(-100), { type: "poll_result",
data: JSON.parse(e.data) }]);
        });
        source.addEventListener("test_started", (e) => {
            setEvents((prev) => [...prev.slice(-100), { type:
"test_started", data: JSON.parse(e.data) }]);
        });
        source.addEventListener("test_completed", (e) => {
            setEvents((prev) => [...prev.slice(-100), { type:
"test_completed", data: JSON.parse(e.data) }]);
        });
        source.addEventListener("manual_confirmation_needed", (e) => {
            setEvents((prev) => [...prev.slice(-100), { type:
```

```

"manual_confirmation", data: JSON.parse(e.data) }]);
});
source.addEventListener("device_discovered", (e) => {
  setEvents((prev) => [...prev.slice(-100), { type:
"device_discovered", data: JSON.parse(e.data) }]);
});
source.addEventListener("worker_health", (e) => {
  setEvents((prev) => [...prev.slice(-100), { type:
"worker_health", data: JSON.parse(e.data) }]);
});

  return () => source.close();
}, [url]);

return { events, connected };
}

// frontend/src/hooks/useApi.js
const API_BASE = "/api/v1";

export function useApi() {
  const apiKey = localStorage.getItem("api_key") || "";

  const headers = {
    "Content-Type": "application/json",
    "X-API-Key": apiKey,
  };

  const get = async (path) => {
    const resp = await fetch(`${API_BASE}${path}`, { headers });
    if (!resp.ok) throw new Error(`${resp.status}`);
    return resp.json();
  };

  const post = async (path, body) => {
    const resp = await fetch(`${API_BASE}${path}`, {
      method: "POST", headers, body: JSON.stringify(body),
    });
    if (!resp.ok) throw new Error(`${resp.status}`);
    return resp.json();
  };

  const patch = async (path, body) => {
    const resp = await fetch(`${API_BASE}${path}`, {
      method: "PATCH", headers, body: JSON.stringify(body),
    });
    if (!resp.ok) throw new Error(`${resp.status}`);
    return resp.json();
  };
}

```

```
    return { get, post, patch };
}
```

main.py — Entry Point

```
import asyncio
from src.config import load_config
from src.influx_writer import InfluxWriter
from src.attestation import AttestationEngine
from src.workers.modbus_worker import ModbusWorker
# import other workers similarly
from src.orchestrator import Orchestrator
from src.test_engine import TestEngine
from src.api import server
import uvicorn

async def main():
    config = load_config()

    # Shared infrastructure
    influx = InfluxWriter(config.influxdb_url, config.influxdb_token,
                          config.influxdb_org, config.influxdb_bucket)
    attestation = AttestationEngine(config)
    event_queue = asyncio.Queue()

    # Test engine and orchestrator
    test_eng = TestEngine(config, influx, attestation, event_queue)
    orch = Orchestrator(config, test_eng, event_queue)

    # Workers
    modbus = ModbusWorker(config, influx, attestation, event_queue)
    # bacnet, snmp, nvml workers similarly

    # Init API server
    server.init(config, test_eng, orch, attestation, influx,
               event_queue)

    # Launch all tasks
    await asyncio.gather(
        attestation.run(),                # attestation consumer
        attestation.wal_recovery_loop(), # WAL recovery
        modbus.run(),                    # modbus polling loop
        # bacnet_worker.run(),
        # snmp_worker.run(),
        # nvml_worker.run(),
        orch.run(),                      # test orchestrator
        uvicorn.Server(
            uvicorn.Config(server.app, host="0.0.0.0",
                           port=config.api_port)
```

```

        ).serve(),                                # API server
    )

if __name__ == "__main__":
    asyncio.run(main())

```

Key: `uvicorn.Server(...).serve()` is the async way to run uvicorn inside an existing event loop. Do NOT use `uvicorn.run()` — that creates its own event loop and blocks.

Module 22: Digital Twin Simulator — `simulator/sim.py`

```

from pymodbus.server import StartAsyncTcpServer
from pymodbus.datastore import import ModbusSlaveContext, ModbusServerContext
from pymodbus.datastore import import ModbusSequentialDataBlock
import asyncio
import math

```

```

class UPSSimulator:
    """Simulates a UPS with dynamic voltage transient on battery
    transfer."""

    def __init__(self):
        # Initialize holding registers (address 0-99)
        # Reg 0: ups_status (2=online, 3=battery)
        # Reg 1: output_voltage (scaled x10, so 4800 = 480.0V)
        # Reg 2: battery_percent
        # Reg 3: battery_test_initiate (write 1 to start, 0 to stop)
        initial = [0] * 100
        initial[0] = 2      # online
        initial[1] = 4800   # 480.0V
        initial[2] = 95     # 95% battery
        initial[3] = 0      # no test

        self.store = ModbusSlaveContext(
            hr=ModbusSequentialDataBlock(0, initial),
        )
        self.context = ModbusServerContext(slaves=self.store,
single=True)
        self.test_active = False
        self.test_start_time = 0

    async def physics_loop(self):
        """Update registers based on simulated physics at 100ms
        intervals."""
        while True:
            hr = self.store.store["h"]  # holding registers

            # Check if battery test was commanded

```

```

test_cmd = hr.getValues(3, 1)[0]

if test_cmd == 1 and not self.test_active:
    # Battery transfer initiated
    self.test_active = True
    self.test_start_time = asyncio.get_event_loop().time()
    hr.setValues(0, [3]) # status = battery

if test_cmd == 0 and self.test_active:
    # Return to utility
    self.test_active = False
    hr.setValues(0, [2]) # status = online
    hr.setValues(1, [4800]) # voltage back to 480.0V

if self.test_active:
    elapsed = asyncio.get_event_loop().time() -
self.test_start_time
    # Simulate voltage transient: dip to 460V in first
5ms, recover to 475V
    if elapsed < 0.005:
        voltage = 4600 # 460.0V dip
    elif elapsed < 0.05:
        # Exponential recovery
        voltage = int(4600 + 200 * (1 - math.exp(-elapsed
* 100)))
    else:
        voltage = 4750 # settled on battery at 475.0V

    hr.setValues(1, [voltage])

    # Battery drains
    battery = hr.getValues(2, 1)[0]
    if battery > 0:
        hr.setValues(2, [max(0, battery - 1)]) # drain 1%
per cycle (accelerated)

    await asyncio.sleep(0.1)

async def run_simulator():
    sim = UPSSimulator()
    server_task = StartAsyncTcpServer(
        context=sim.context,
        address=("0.0.0.0", 502),
    )
    await asyncio.gather(server_task, sim.physics_loop())

if __name__ == "__main__":
    asyncio.run(run_simulator())

```


Key: PyModbus 3.x server API. StartAsyncTcpServer with ModbusServerContext. The physics loop runs alongside the server in the same event loop, updating the ModbusSequentialDataBlock registers directly. The platform connects to port 502 and sees a UPS that behaves dynamically.

Registers are 0-based in the datastore. When the platform reads address=1 (output_voltage), PyModbus maps it to index 1 in the ModbusSequentialDataBlock.

What This Guide Does NOT Include

- Module 9 (NVML): pynvml is straightforward. nvmlInit(), nvmlDeviceGetHandleByIndex(i), nvmlDeviceGetTemperature(handle, NVML_TEMPERATURE_GPU). Wrap each call, return PollResult.
- Module 11 (Discovery): asyncio.gather with semaphore over IP range. Per-IP: try Modbus connect on 502, SNMP get sysDescr on 161, BACnet Who-Is on 47808. Classify based on response.
- Module 12 (REST Connectors): httpx.AsyncClient.get() against Genetec/ServiceNow API. Parse JSON response. Normalize to AttestationRecord. Submit to attestation queue. 100 lines per connector, all the same pattern.
- Module 13 (Orchestrator): DAG from NetBox power connections. asyncio.Queue consumer. Check conflicts against running tests. Dispatch to test_engine.execute(). ~200 lines.
- Module 14 (Reconciliation): Query NetBox for all devices. Compare against design (also in NetBox). Build PunchListItem for each mismatch per the algorithm in the contracts spec. ~200 lines.
- Module 15 (Reports): reportlab PDF generation from PunchListItem list + attestation chain summary. ~300 lines.
- Module 16 (BIM Import): ifcopenshell.open(path). Iterate IFC entities. Map IfcDistributionElement to NetBox device objects via API. ~200 lines.
- Module 17 (PDF Pipeline): Read PDF with pdfplumber. Send text to Anthropic API with Config Context schema in system prompt. Parse JSON response. Validate against schema. Load via Module 1. ~300 lines.
- Modules 18-20 (React pages): Standard React components calling useApi() hooks. Each page maps to one API section from the contracts spec. The Dashboard is the largest at ~1500 lines.

These are omitted because they're either trivially straightforward (REST connectors, NVML), or require only the contracts spec + the hooks/API patterns shown above to implement correctly.